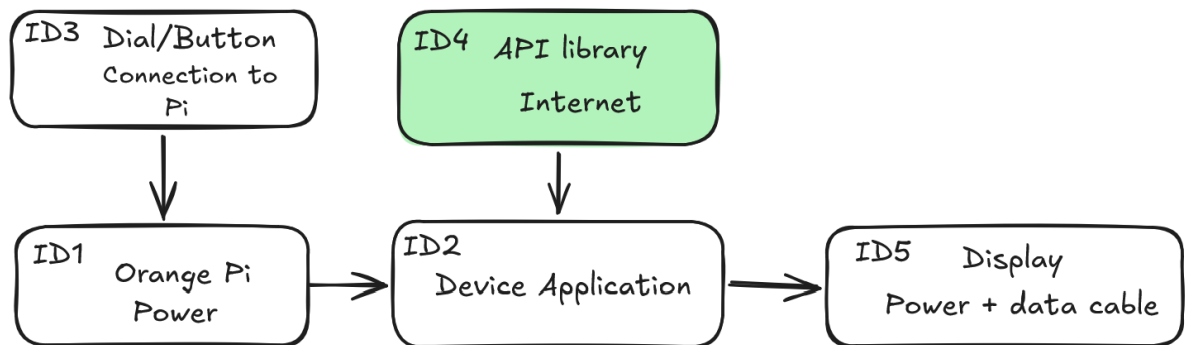


1) Title, goal statement, conventions (for diagrams)

2) D0 diagram:

<https://excalidraw.com/#room=59d14f1851a736a71b5e,nkBUBcKHbZdds4uSyUWQJA>

- a. Orange Pi 3 (power)
- b. Dial buttons (input)
- c. APIs (internet)
- d. Device Applications
- e. Display (power, display input)



3) Component responsibility table

Component	Description	Input	Output	Responsibility
ID1	Manages IO and runs software	Dial Buttons	Digitalized Button information	Elliot Warner
ID2	Manipulates data into desired format	Button Information Data from API	Display Instructions	William Thomas
ID3	Handles user input	User Input	Electronic dial information	Elliot Warner
ID4	Retrieves App Data	API requests	Information from various sources	Nate Heath
ID5	Displays to apps to user	Display Instructions	Visual Display on a Screen	Nate Heath

4) Interface specification table

Connection	Input	Output	Data Format	Protocol	Error	Responsibility
Users Local Machine	User inputs via USB cable/Internet	JSON Showing Successful upload	Files in python	HTTP Post	Error thrown on computer showing user that upload was failed	Elliot
Cleansing Logic	Python File	Sanitized Python File And JSON showing Success	Python File (Text file)	Internal Python Function	Output to Console/error log	Nate
Server	Sanitized Python File	Python File Hosted By Orange Pi	(Not yet Defined)	(Not Yet Defined)	Output to Console/error log	Nate
Orange Pi Device	Hosted Python file and input from dials	LED formatted data for display	RGB Pixel Array Stream	(Not Yet Defined)	Output log to error file	Will
Physical Dial Input	User input to Dials	Signal of movement	Usage IDs	GPIO SPO bus polling	Error Handling on display	Will
LED Display	LED Instructions	Lights activation to form display	(Not Yet Defined)	HUB75 Board Interface	No Display or all RED	Elliot

Payload Example JSON Returned from File:

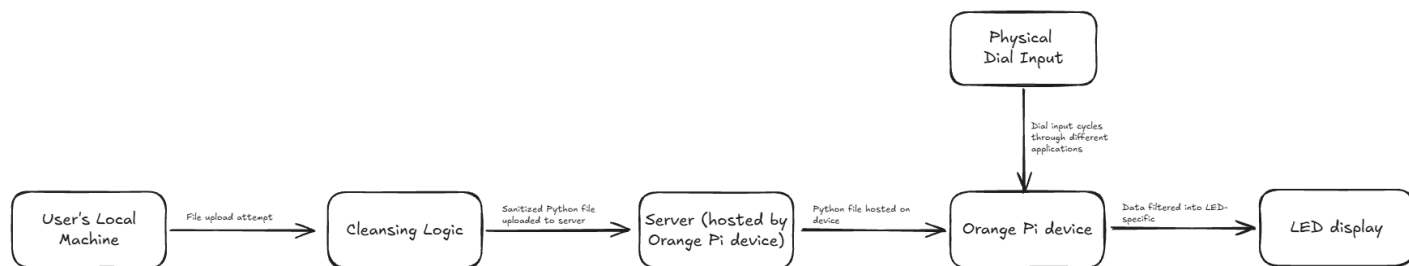
```
{
```

```

"status": "success",
"filename": "counter_app.py",
"file_size_bytes": 1042,
"sanitization_passed": true,
"assigned_slot": 2,
"timestamp": 1759432800
}

```

Data-flow diagram



5) Architecture pattern and justification

a. Application upload/management

This part of the system is designed to allow a user to visit an endpoint to upload their application files through a user interface. The **client-server architecture** best fits our problem shape, as we can have the Orange Pi host a small server for the user, on their client device, upload the file to. The Orange Pi will then take the received file and run it on its hardware. Our team has plenty of experience building out servers, clients, and network interaction, both from co-ops and classes. Since the server will be small and simple, the Orange Pi's hardware should be able to host it without any issues. Lastly, since we expect each Orange Pi's personal server to only be accessed by one or two devices at a time, performance/timing/scalability are not concerns.

While the layered architecture pattern is additionally valid, this part of the system requires very little business logic and a very simple UI, to the point that implementing a layered approach would simply bloat the code required

for this feature. Additionally, the layered approach places a smaller emphasis on networking, which is the most important part of this feature.

b. Application code sanitization

Our system requires us to enforce security practices on the user's application code to ensure that it doesn't contain malicious code that could leak information or break the Orange Pi. Therefore, we must apply a **pipeline / data-flow architecture** to break the application code upload into multiples stages. This way, we can insert a stage in the pipeline before the code is uploaded to the server where we sanitize the code to ensure it has no ill-intent. This use case fits the idea behind the pipeline architecture: moving/piping data from one place to another while modifying it as needed. Our team has much experience building data pipelines, as we have all worked with large software (both frontend and backend) in previous co-ops. Performance and timing are not a concern with this use case, as most application files will be relatively small and the code sanitization algorithm(s) will be simple. The same insight we shared about the application upload above remains for the code sanitization: we only expect one or two devices to access each server at a time, so scalability is not a concern. Again, we could have applied a layered architecture pattern to this problem and had the business logic layer handle all of the sanitization, but this process is simple enough that a layered architecture would only bloat the solution.

c. Dial button input

Our design for the physical product contains two small dial buttons on the device. One dial will swap between applications to allow you to load a separate application onto the display. The other dial will be for use by the user's program, to provide a simple form of input for them to build their program around. For simple dial button inputs, we deemed it best to use the **embedded / sensor-actuator architecture**, as this architecture is best for mapping simple inputs to processes, and awaiting another input. In this case, the user is going to write most of the logic since they define the process that will run on dial button input, so our architecture will be even simpler. Additionally, this architecture is used in robotics, so we can safely assume that our performance and timing needs will be met. The input cannot be scaled, as you only have a negative and positive direction to twist the dial. This means we have little to no scalability needs. Lastly, since the dial button

input is wired directly into hardware, we see this architecture as the best-fitting.

We could have chosen to use a pipeline / data-flow pattern for this problem, but the dial button input data is not being transformed and wouldn't pass through many processing stages. It simply maps to user-defined functions.

6) Decision Log

a. Raspberry Pi vs. competitors

Here we were deciding on what we wanted to use to run our software. We were deciding between the Raspberry Pi Zero 2W and the Orange Pi Zero 3. We ultimately decided on the Orange Pi Zero 3 due to its availability and price. The Orange Pi Zero 3 is much easier to find at a reasonable price and outperforms its Raspberry Pi equivalent.

b. Screen type

For the screen type, we were deciding between a 64x32 LED RGB matrix or an eink display. We were considering the eink display due to its power efficiency, detail, and affordability but decided on the LED matrix. The main reason we landed on the LED matrix was the fact that the eink display struggles without a light source. A user may find it difficult to use this display at night, as it can only show the time.

c. Screen size/pitch

After landing on the 64x32 LED RGB matrix, we needed to figure out our desired pitch (distance between pixels). Oddly, we weren't very concerned with detail as much as the pure size. We considered 2.5mm, 3mm, and 4mm. We decided to go with a 3mm pitch, since the width of the display is roughly 7.5 inches. The others were 6.3 inches and 10 inches, respectively. 7.5 inches made the most sense for us.

d. Speaker vs. no speaker?

The only two options here were speaker or not. We are planning to go with no speaker, as the Orange Pi 3 has Bluetooth. We don't believe that we would be able to fit a suitable enough speaker in the chassis, nor would it be necessary for our current vision.